

Module 09: Sealed Types and Pattern Matching

This module is where three features you have already met stop being separate things. Sealed interfaces say what the cases are, records say what data each case carries, and an exhaustive switch says what to do with them. The compiler checks that all three agree.

The result is what other languages call algebraic data types, written in ordinary Java.

What You'll Learn

- `sealed`, and the three modifiers every permitted subtype must choose from
- Pattern matching with `instanceof`, and the cast it removes
- Switch patterns, guards with `when`, and explicit `case null`
- Record patterns, including nested destructuring
- Why exhaustiveness is the whole payoff, and what `default` costs you

Coming From Another Language

	Elsewhere	Java
Closed set of cases	Rust <code>enum</code> , Kotlin <code>sealed</code> , TS union	<code>sealed interface</code>
Destructuring	Python <code>match</code> , JS destructuring	record patterns
Guard	<code>if</code> inside the arm	<code>when</code> clause
Exhaustiveness	Rust enforces it	enforced for sealed types

If you have written Rust or Kotlin this will feel immediately familiar. If you have written older Java, this is the feature that most changes how you model data.

The Lesson

sealed

```
final      nobody may extend this
sealed    only these named types may extend this
(neither) anybody, anywhere, forever
```

Module 06 argued for `final` by default. `sealed` is the answer when you *do* want subtypes, but a known, fixed set:

```
sealed interface Shape permits Circle, Rectangle, Triangle { }
```

Every permitted subtype must declare its own intent, choosing one of `final`, `sealed`, or `non-sealed`. Leaving all three off is a compile error:

```
error: sealed, non-sealed or final modifiers expected
```

Records are implicitly final, which is why a record needs no modifier. That pairing is deliberate, and it is the shape most modern Java data modelling takes.

An uninvited type gets refused:

```
error: class is not allowed to extend sealed class: Result (as it is not listed in its
'permits' clause)
record Maybe(String hint) implements Result {
^
```

Seal when the set of cases is part of the design. A payment is a card, a transfer or a voucher. A parse either succeeded or it did not. In those, "someone might add another later" is a risk you want caught, not an extension point.

Do not seal a plugin interface. Sealing something third parties should implement is how you ship a library nobody can extend.

Pattern matching

`case String s` tests the type, casts, and names the result, all at once. From `PatternMatching.java`, which writes the same logic twice:

```
// modern
case String s when s.isEmpty() -> "an empty string";
case String s -> "text of length " + s.length();

// old
if (o instanceof String) {
    String s = (String) o;
    return s.isEmpty() ? "an empty string" : "text of length " + s.length();
}
```

The old form names the type twice and performs a separate cast. That cast is where bugs lived: nothing stops you writing `if (o instanceof String) { Integer i = (Integer) o; }`, which compiles and throws.

Three details worth knowing:

- **Guards use `when`**, and guarded cases must come *before* the unguarded case for the same type. First match wins, so an unguarded `case String s` placed first would swallow everything.
- **`case null` is explicit.** Without it, switching on null throws `NullPointerException`, the pre-existing behaviour kept for compatibility.
- **`instanceof` patterns work outside switch**, which is where you will use them most:

```
if (o instanceof String s && s.length() > 5) {
    return s.substring(0, 5) + "...";
}
```

The scoping is smarter than it looks. After an early return, the variable is in scope for the rest of the method:

```
if (!(o instanceof String s)) {
    return -1;
}
return s.length(); // s is in scope here
```

Record patterns

Destructuring in the pattern itself. From `RecordPatterns.java`:

```
case Circle(Point(var x, var y), var r) when x == 0 && y == 0 ->
    "circle of radius " + r + " at the origin";

case Rectangle(Point(var x1, var y1), Point(var x2, var y2))
    when (x2 - x1) == (y2 - y1) ->
    "a square of side " + (x2 - x1);
```

Real output:

```
circle of radius 5.0 at the origin
a square of side 4
a 6 by 3 rectangle
```

Detecting a square needed no method on `Rectangle` at all. The pattern matched the type, pulled out two nested points, bound four values and compared them, in one case label.

Exhaustiveness is the payoff

A switch over a sealed type needs no `default`, because the compiler can prove it is complete. Leave a case out and `NotExhaustive.java` happens:

```
error: the switch expression does not cover all possible input values
    return switch (payment) {
           ^
missing patterns:
    Voucher _
```

It names the missing type. The `Voucher _` is an unnamed pattern: the compiler is telling you a branch that ignores the value would satisfy it.

This is the whole argument for sealing. Add a constant to the `permits` clause and every incomplete switch in the entire codebase becomes a build failure that points at itself. Writing `default -> 0` instead

would have shipped a voucher with no fee and no warning.

Module 03 made this case for enums. Sealed types extend it to any closed set of shapes, with data attached.

Run It

```
java modules/09-sealed-and-pattern-matching/examples/SealedTypes.java
java modules/09-sealed-and-pattern-matching/examples/PatternMatching.java
java modules/09-sealed-and-pattern-matching/examples/RecordPatterns.java

# Both fail on purpose. Read which type each one names.
java modules/09-sealed-and-pattern-matching/examples/NotExhaustive.java
java modules/09-sealed-and-pattern-matching/examples/NotPermitted.java

./scripts/verify-examples.sh modules/09-sealed-and-pattern-matching
```

Common Mistakes

Adding `default` to a switch over a sealed type. You have just turned every future compile error into a silent wrong answer. This is the single most important habit in this module.

Putting the unguarded case first. `case String s` before `case String s when ...` makes the guarded branch unreachable, and the compiler says so.

Forgetting `case null`. A switch on a null reference throws unless you list it. If null is possible, handle it visibly.

Sealing an interface third parties need to implement. Sealed is for closed sets you own, not for extension points.

Declaring a permitted subtype with no modifier. Every one must say `final`, `sealed` or `non-sealed`. Records already say `final`.

Reaching for deep inheritance where a sealed hierarchy fits. If the subtypes differ in data rather than behaviour, a sealed interface of records plus a switch is usually clearer than an abstract class with overrides.

Key Takeaways

- `sealed` names the complete set of subtypes, and every one of them must declare `final`, `sealed` or `non-sealed`.
- **Patterns test, cast and bind in one step**, removing the cast that used to go wrong.
- **Guards use `when` and must precede the unguarded case** for the same type.

- `case null` is explicit; otherwise a null throws.
- **Record patterns destructure, and they nest** as deep as the data.
- **Omit `default` over a sealed type.** The missing-case compile error is the feature you are paying for.
- **Sealed interface + records + exhaustive switch** is how modern Java models a closed set of data shapes.

Homework

[homework/README.md](#)

Model an expression evaluator as a sealed hierarchy, then prove the compiler catches a forgotten case.

Reference solution in [solutions/09-sealed-and-pattern-matching/](#).